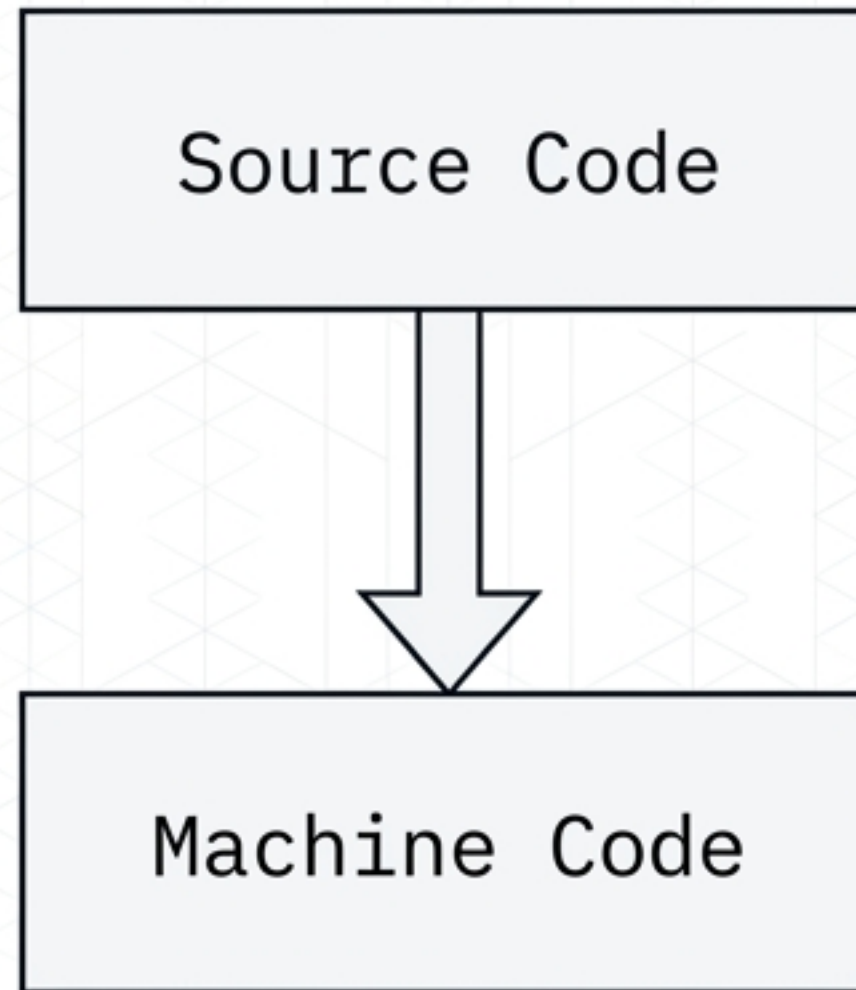


Fundamentos das Linguagens de Programação

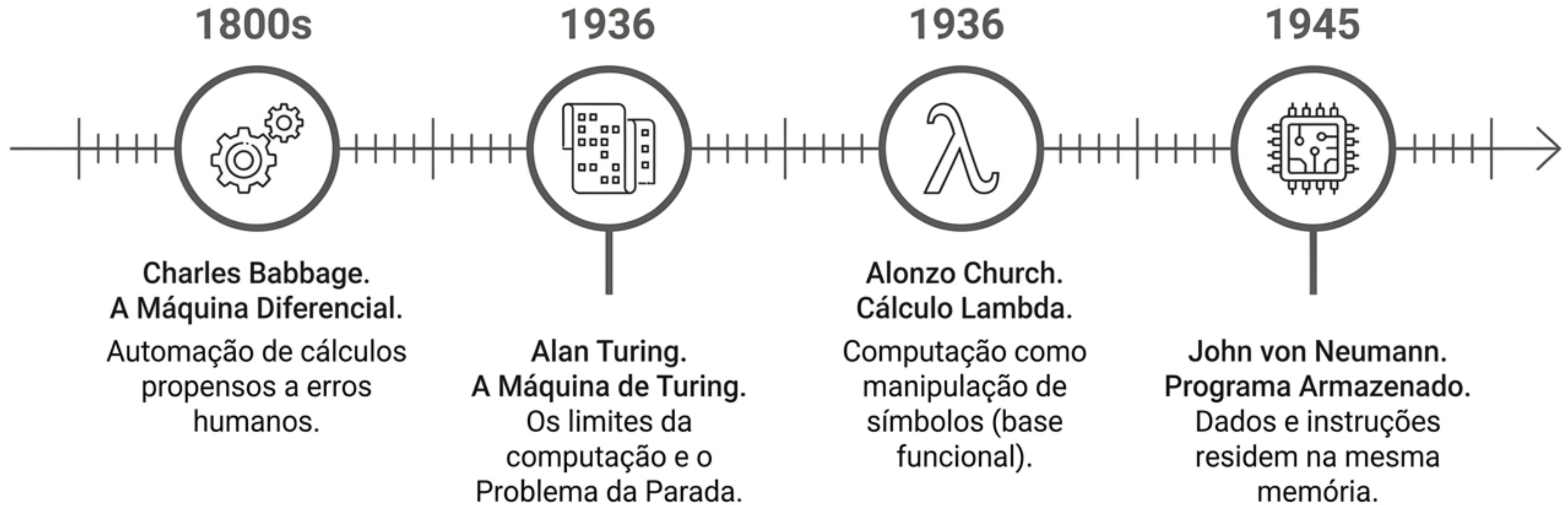
Paradigmas, Modelos e Execução: Um guia visual passo a passo



// Do Conceito à Execução

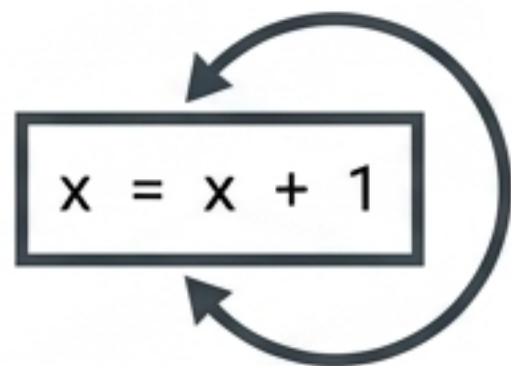
Matemática

Ciência da Computação



Os Três Pilares da Programação

Modelo Imperativo / OO

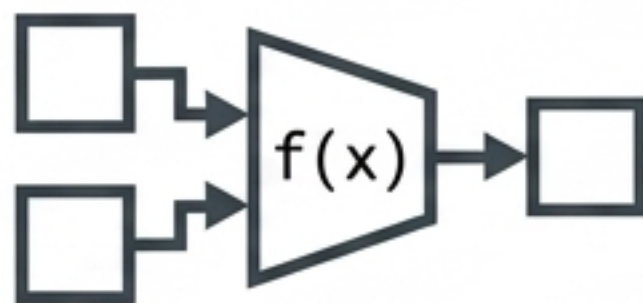


Foco: Alteração de estado e variáveis.

O programador dita *como* a memória deve ser atualizada passo a passo.

Java, C++, Python

Modelo Funcional

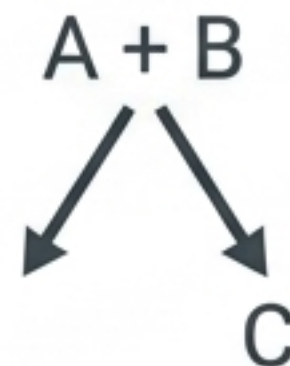


Foco: Funções e imutabilidade.

Constrói novos valores a partir dos antigos, sem alterar o estado original.

Standard ML, Haskell, Lisp

Modelo Lógico

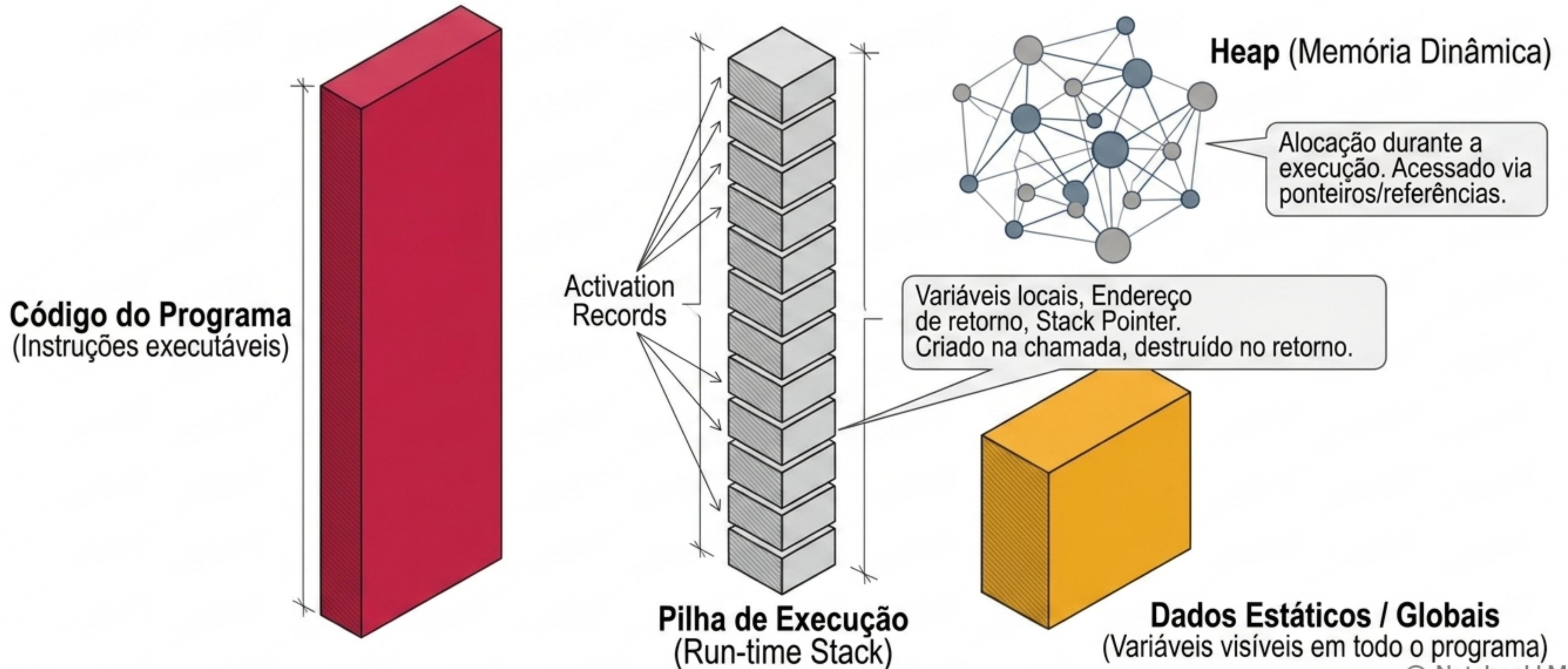


Foco: Regras, fatos e deduções.

O programa é um banco de dados que responde perguntas lógicas.

Prolog

O Modelo Imperativo: Arquitetura da Memória



O Modelo Funcional: A Revolução Imutável

Cidadãos de Primeira Classe

A divisão entre Programa e Dados desaparece. Funções são tratadas como dados e passadas como parâmetros.

Código do Programa (Instruções executáveis)

Vantagem Cognitiva

O programador ignora a arquitetura subjacente de von Neumann para focar no fluxo matemático.

Activation
Records

Pilha de Execução
(Stack)

Variáveis locais, Endereço
de retorno, Stack Pointer.
Criado na chamada, destruído

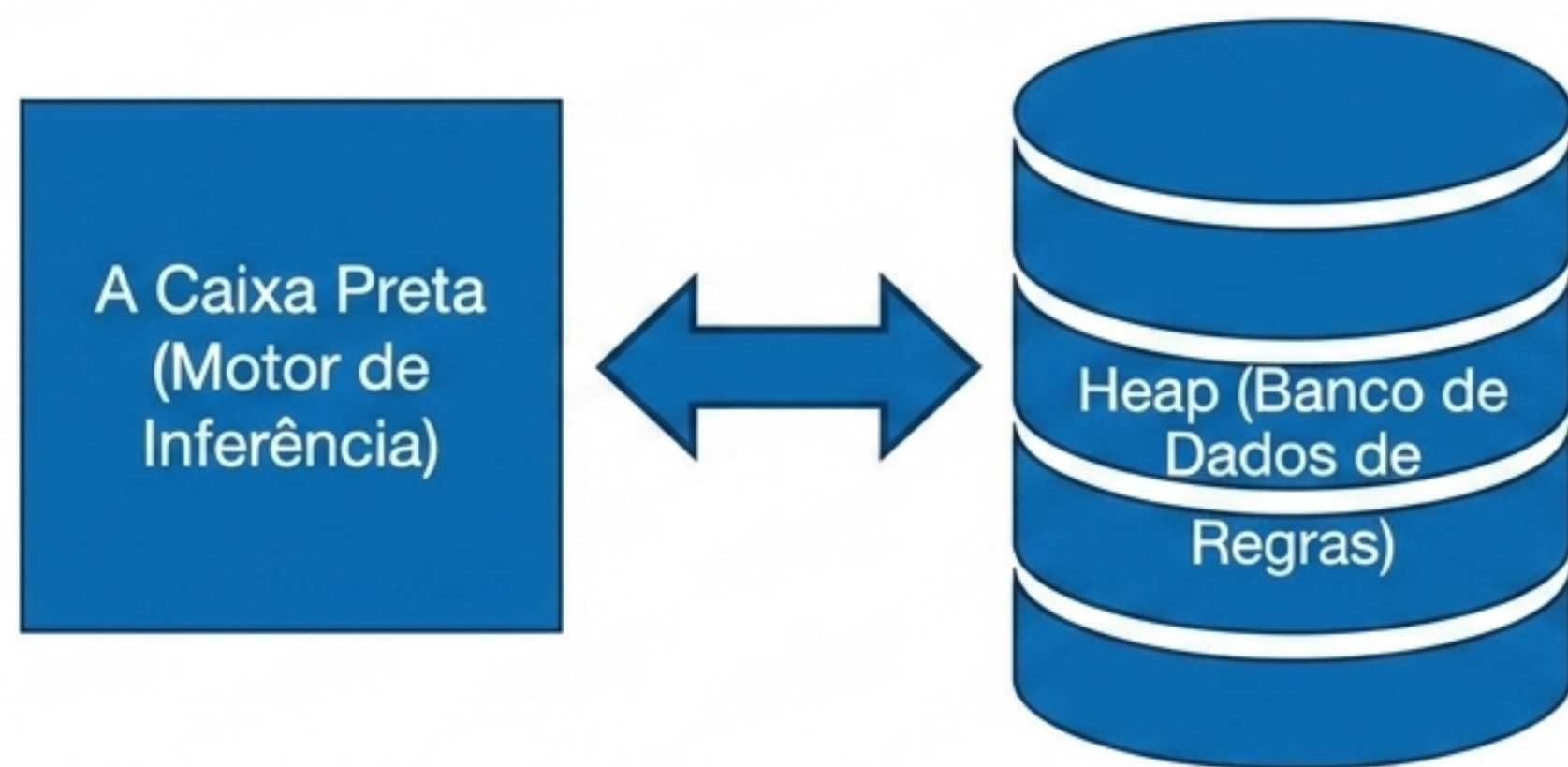
Heap (Memória Dinâmica)

Alocação durante a
execução. Acessado via
ponteiros/referências.

Minimização do Heap
Dados não sofrem mutação
(imutabilidade). Novos
valores são sempre
construídos a partir de
velhos.

Dados Estáticos / Globais
(Variáveis visíveis em todo o programa)

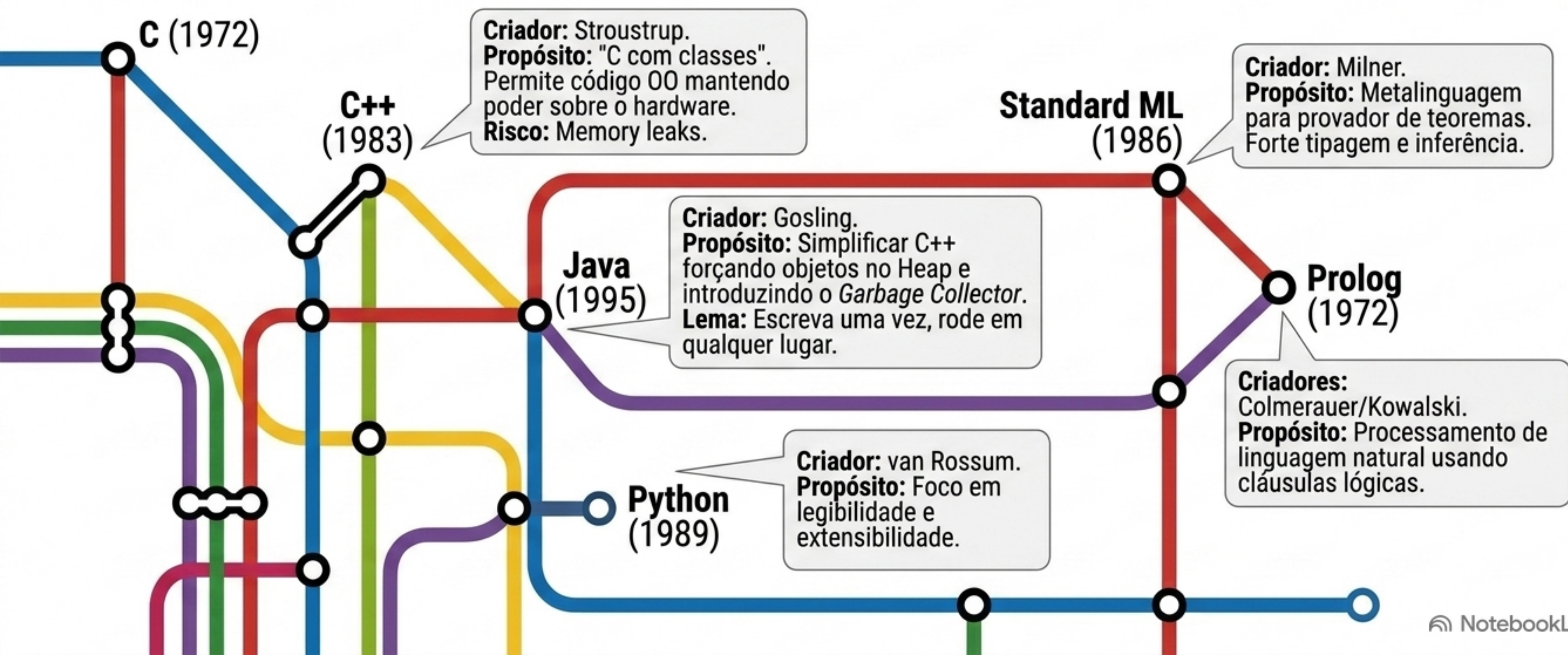
O Modelo Lógico: A Caixa Preta



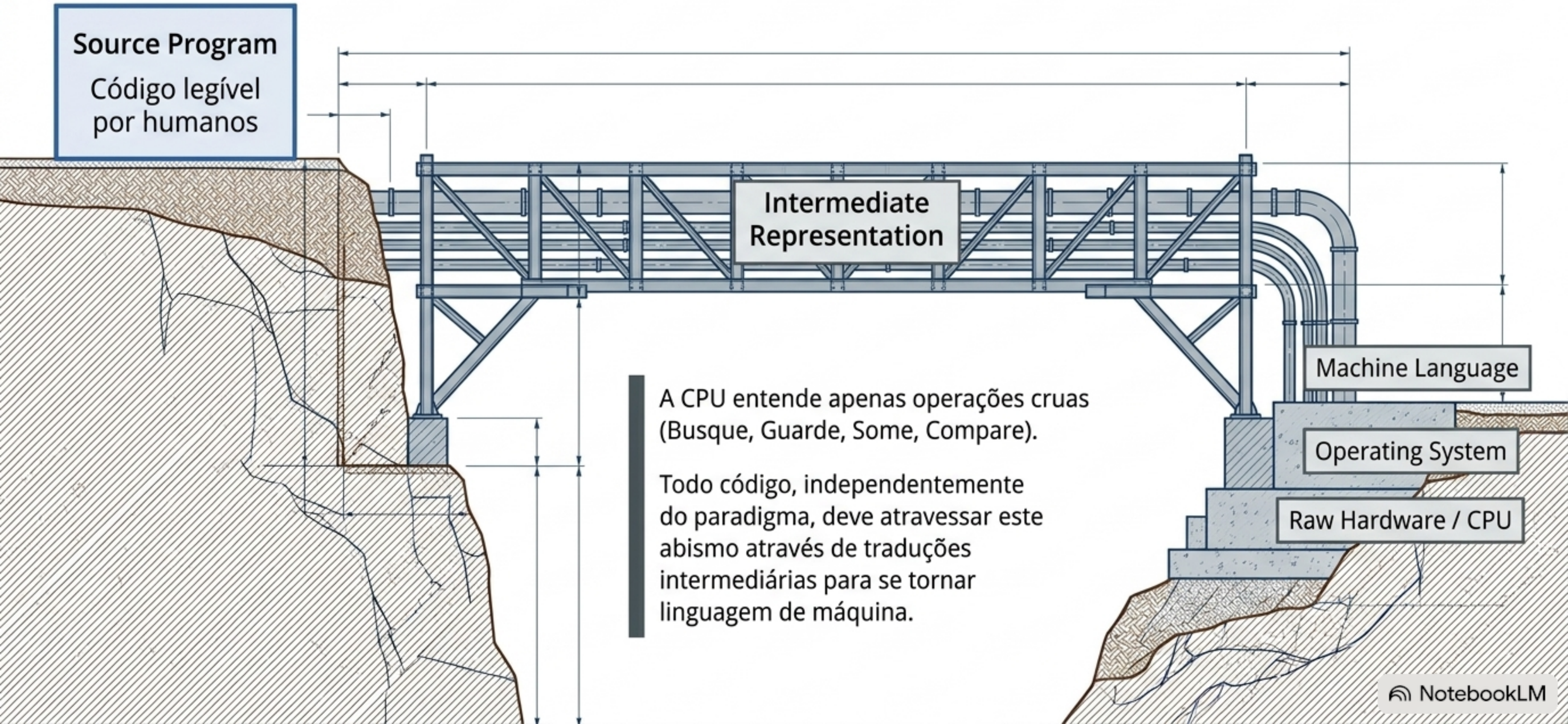
- **A Abstração Total:** A pilha e o heap tradicionais existem no hardware, mas são abstraídos por uma Máquina Virtual.
- **Foco no Problema, não nos Passos:** Em Prolog, você não escreve a solução. Escreve um banco de dados de *fatos* e *regras lógicas* .
- **Execução:** A linguagem navega pelas regras tentando responder às perguntas do usuário com deduções de Sim ou Não.

A Árvore Genealógica

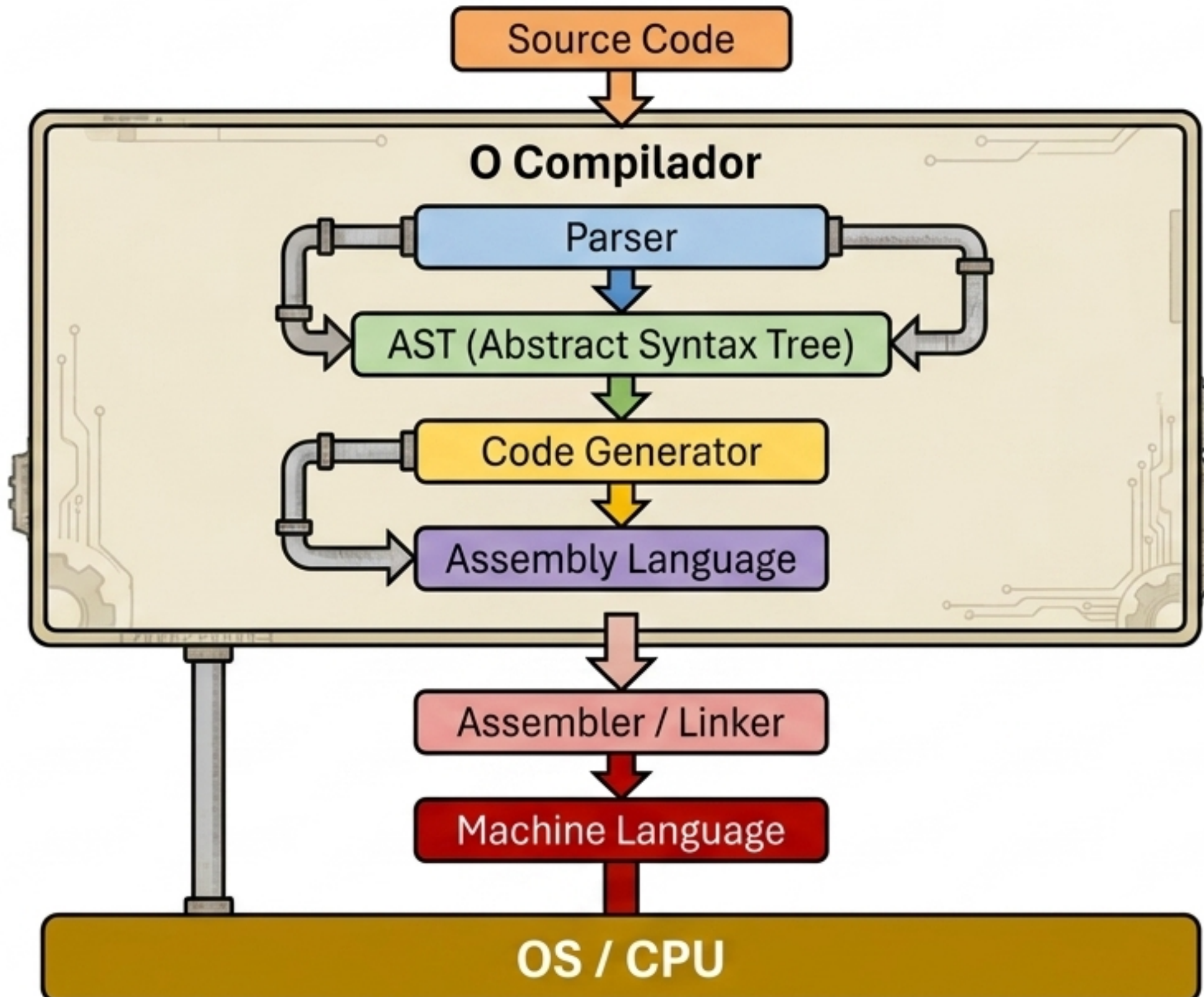
Origens e propósitos das linguagens chave



O Grande Desafio: A Tradução



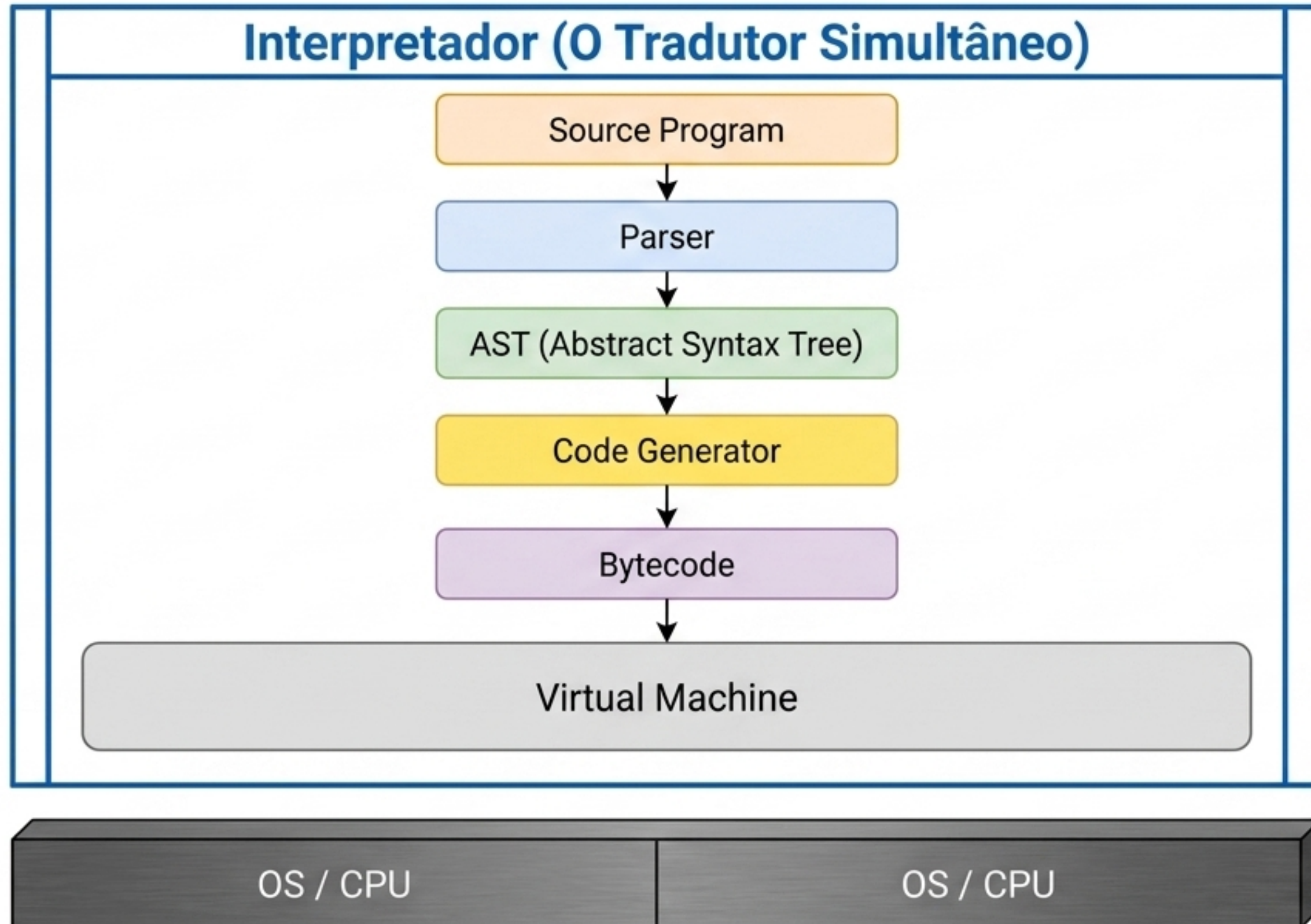
Caminho 1: Compilação Direta



Painel de Status

- **✓ Vantagem:** Execução extremamente rápida (tradução ocorre toda antecipadamente).
- **✗ O Problema:** Bloqueio (Lock-in) de hardware. O binário resultante está preso a um Sistema Operacional específico.
- **Exemplos:**
C, C++

Caminho 2: Interpretação



A Engrenagem em Tempo Real:
O código é traduzido linha por linha durante a execução.

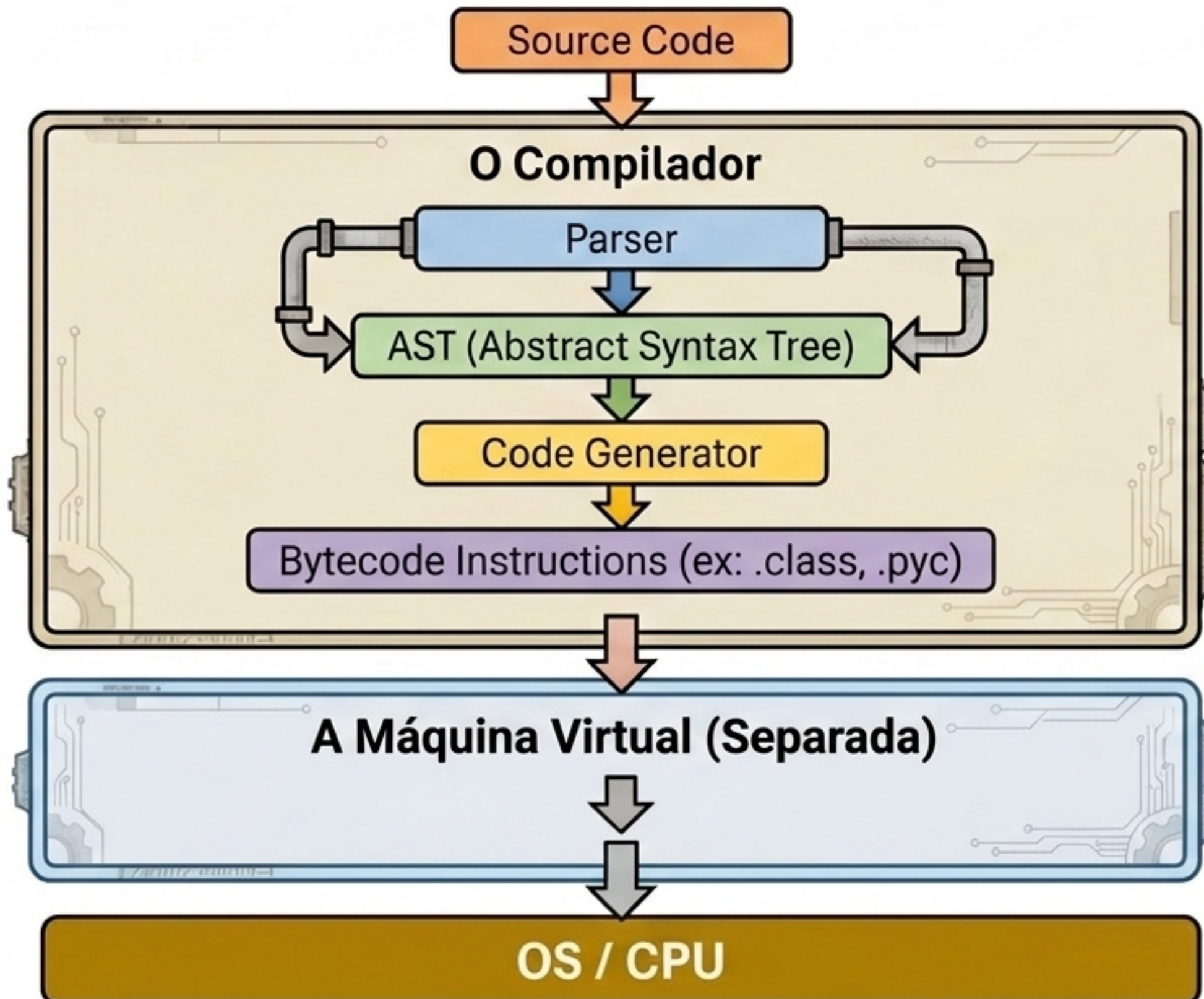


Gestor de Memória: O Interpretador gerencia o *Garbage Collector* em segundo plano.



Trade-off:
Roda em qualquer lugar que tenha o interpretador instalado, mas é inerentemente mais lento.

Caminho 3: Máquinas Virtuais (Híbrido)



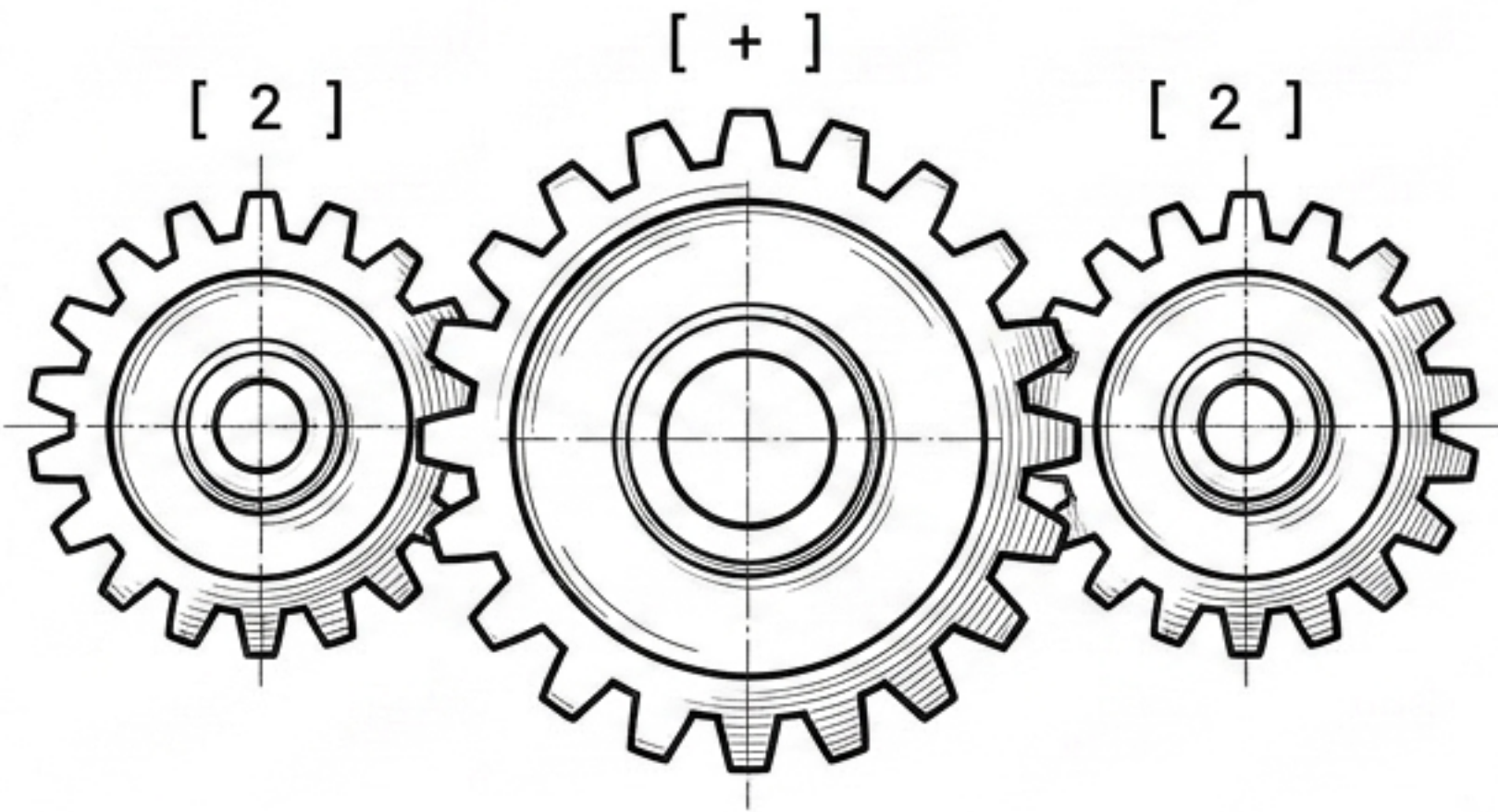
O Melhor dos Dois Mundos

- Tira vantagem da velocidade de uma pré-compilação básica e da imensa portabilidade do interpretador.
- Permite a distribuição de arquivos binários compilados (proteção de IP).
- Usado por Java, C# (.NET) e Python.

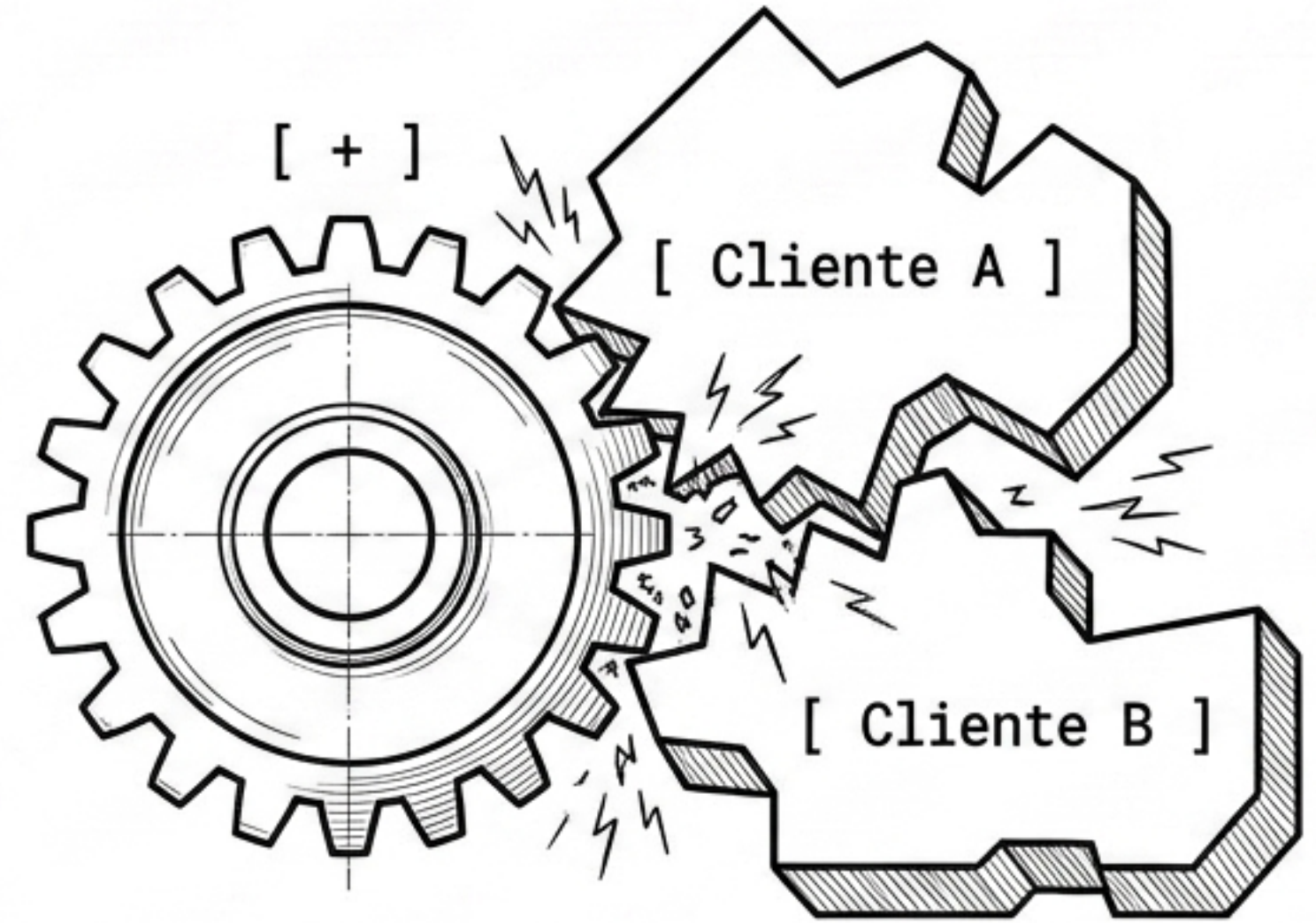
Matriz de Comparação: Implementações

Abordagem	Velocidade	Portabilidade	Gestão de Memória
C/C++ (Compilação Direta)			Alocação Manual (Risco de Memory Leak)
Bash/Ruby (Interpretado Puro)			Garbage Collection (Automático)
Java/C# (Híbrido/VM)			Garbage Collection (Otimizado)
Python (Interpretado + VM)			Garbage Collection (Automático)

O Papel dos Tipos de Dados: Type Checking



Operação Válida. O encaixe mecânico funciona.

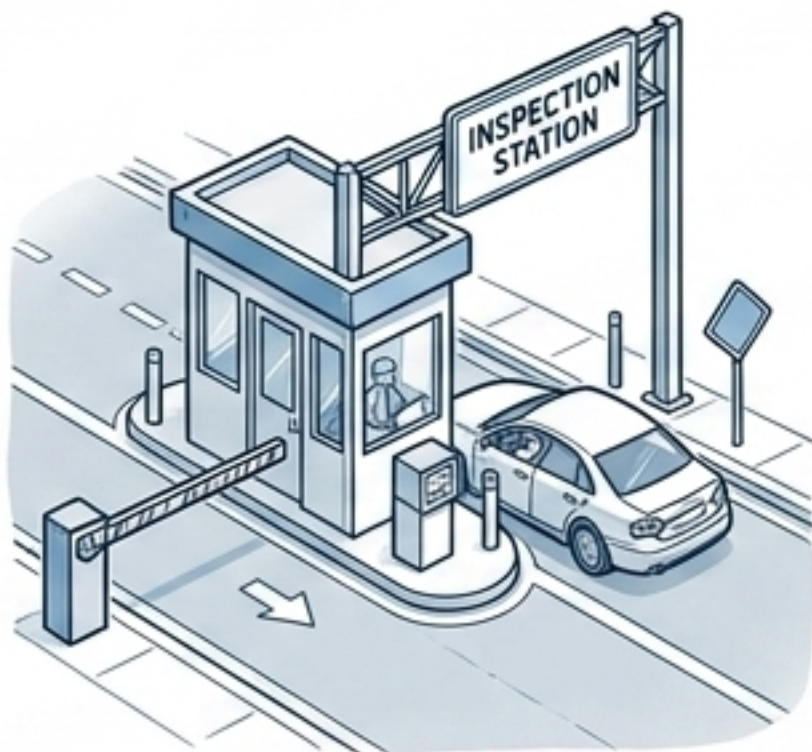


Quebra de Sistema. A soma não tem sentido lógico para tipos "cliente".

A linguagem realiza o Type Checking para verificar se a operação solicitada faz sentido matemático e lógico com a estrutura do dado.

Tipagem Estática vs. Dinâmica

Tipagem Estática



Languages (Monospace): `Java, C++, ML`

O erro é pego **antes** da execução (*Compile-time*).

Exige mais esforço arquitetônico inicial do programador, mas gera softwares blindados com drástica redução de bugs em execução.

Tipagem Dinâmica



Languages (Monospace): `Python, Ruby`

Os tipos nascem com o valor.
O erro é pego no **último segundo** (*Run-time*).

Prototipagem extremamente ágil. Porém, obriga a construção de baterias rigorosas de testes, pois operações ilegais permanecem ocultas até a execução

Conclusão: A Ferramenta Certa

1. Paradigma (A Modelagem)

Abordagens Prescritivas

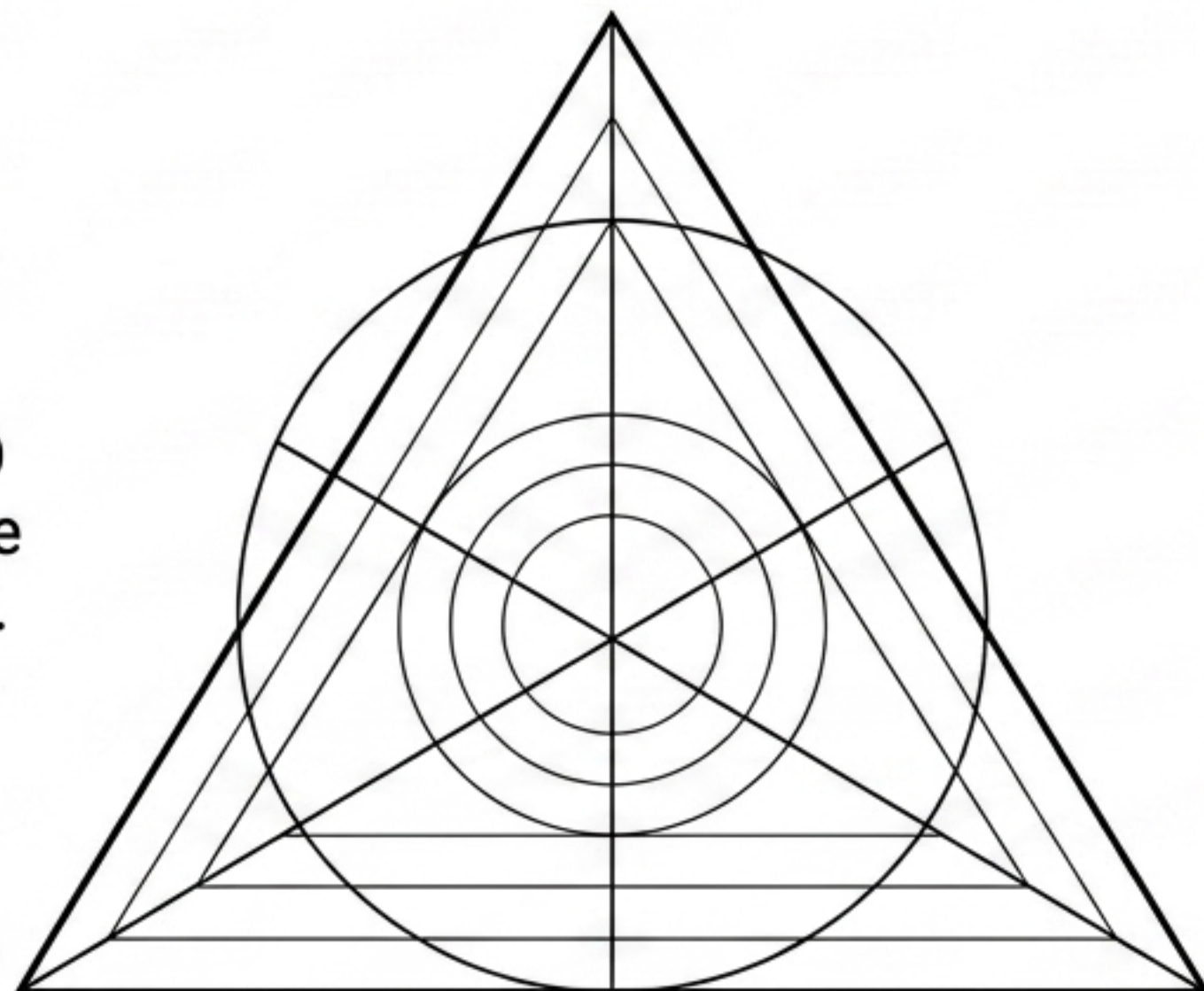
Foco rigoroso nos detalhes microscópicos da máquina. O programador dita o controle de estado e a memória (ex: C++).

Abordagens Descritivas

Foco em descrever relações macro e focar exclusivamente na natureza do problema (ex: Prolog, ML).

3. Tipagem
(A Lógica)

2. Execução
(A Física)



Um engenheiro de software sênior não decora sintaxes; ele compreende os modelos abstratos, a arquitetura de tradução e as regras de tipagem por trás das abstrações.